



CSE461

Introduction to Robotics Lab

Lab No. : 05

Group : 01

Section : 06

Semester : Summer_2025

Group members :

Azmari Sultana	22201949
B M Rauf	22201782
Sayeba Nasir	22201354

Submitted Date: 15-08-2025

Submitted to - CSDM

1. Objectives: The objective of this lab is to introduce students to servo motors and how to control them using a Raspberry Pi microcontroller. The lab is designed to teach the basic function of servo motors, duty cycles, and their working mechanisms. Students will learn to program the GPIO pins to control the motor's rotation to a specific position using PWM (Pulse Width Modulation). The lab also incorporates an ultrasonic sensor, which students will use to control the servo motor based on real-time sensor data. Ultimately, this lab aims to teach students how to build a simple automated system that interacts with its environment and reacts in real-time, such as an automated parking gate.

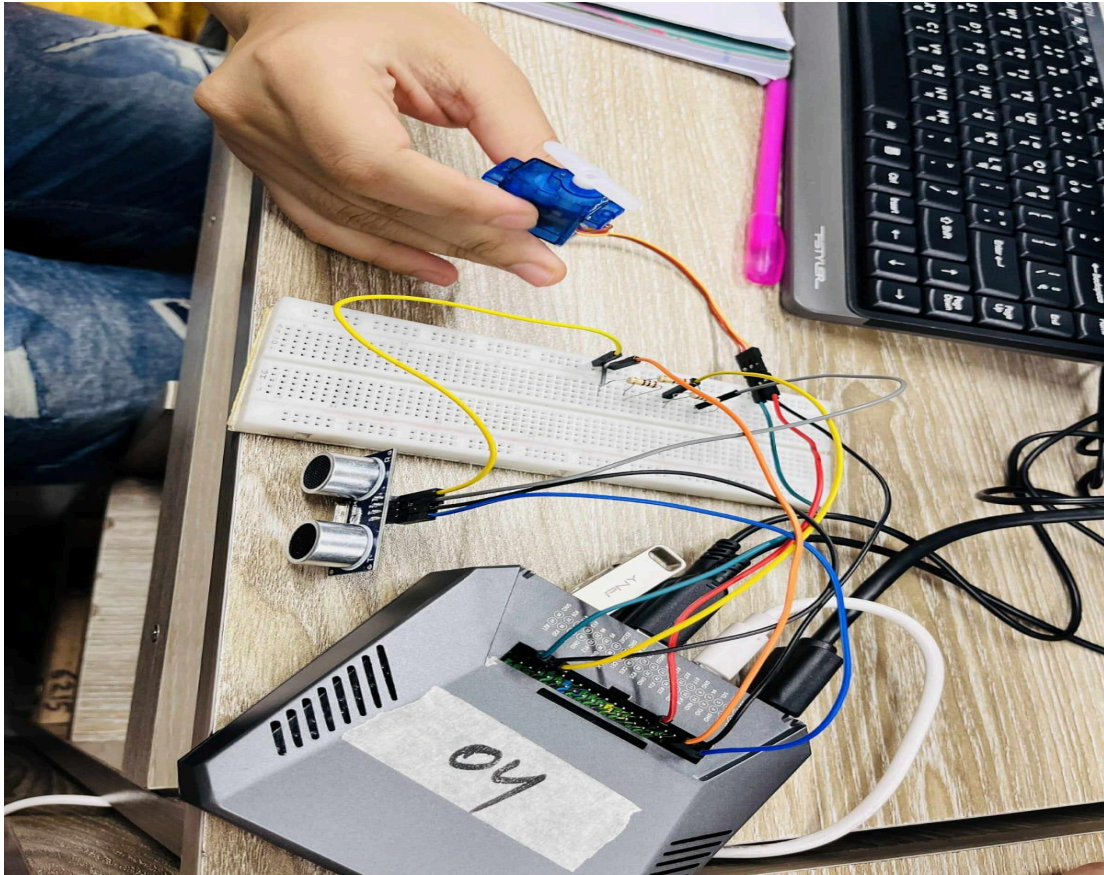
2. Equipments:

Raspberry Pi development board
Servo Motor (specifically, the SG90 Model)
Ultrasonic Sensor (HC-SR04)
Resistor (around 220 ohms)
Jumper wires
Breadboard
USB pen drive with the OS

3. Experimental Setup:

(Picture + Explanation)

The experiment is set up to control a servo motor using a Raspberry Pi and an ultrasonic sensor. The SG90 servo motor has three wires: red, brown, and orange. The red wire is connected to a 5V power pin, the brown wire to a ground pin, and the orange wire to a GPIO pin (GPIO17 is specified) on the Raspberry Pi. The Raspberry Pi sends a PWM signal to the servo motor, with the motor's position determined by the signal's duty cycle. The objective is to program the system so that when the ultrasonic sensor measures a distance less than 5cm, the servo motor rotates to 180°. Otherwise, it rotates to 90°. This setup essentially creates a simple automated parking gate.



4. Code: (If Applicable)

```
# Enter Code Here

import RPi.GPIO as GPIO
from gpiozero.pins.pigpio import PiGPIOFactory
from gpiozero import Device, Servo, AngularServo
from time import sleep
Device.pin_factory = PiGPIOFactory()
s = AngularServo(17, min_angle = 0, max_angle = 180, min_pulse_width=0.5/1000,
max_pulse_width = 25/10000)

import time
GPIO.setmode(GPIO.BCM)
TRIG = 26 #GPIO26
ECHO = 19 #GPIO19

GPIO.setup(TRIG,GPIO.OUT)
GPIO.setup(ECHO,GPIO.IN)

def distance():
```

```

GPIO.output(TRIG, False)
time.sleep(0.5)
GPIO.output(TRIG, True)
time.sleep(0.00001)
GPIO.output(TRIG, False)
pulse_start = time.time()
while GPIO.input(ECHO)==0:
    pulse_start = time.time()
while GPIO.input(ECHO)==1:
    pulse_end = time.time()
pulse_duration = pulse_end - pulse_start
distance = pulse_duration * 17150
distance = round(distance, 2)

    return distance

while True:
    x = distance()
    print(x)
    if x<5:
        s.angle = 180
    else:
        s.angle = 90
GPIO.cleanup()

```

5. Results (Output of the experiment):

The lab's final task is to build an automated parking gate using the servo motor and ultrasonic sensor. The expected output is a system where a servo motor rotates to a 180-degree position when an ultrasonic sensor detects an object closer than 5 cm, and returns to a 90-degree position when the object is further away. The servo motor is controlled by a PWM signal generated by the Raspberry Pi, with the duty cycle changing based on the sensor's input.

6. Discussions/Answers:

A servo motor's position is controlled by the duty cycle of a PWM signal, with a constant frequency of 50Hz. The pulse width, or "ON" time, of the signal determines the angle of rotation. A 1ms "ON" time results in a 0° rotation, 1.5ms for 90°, and 2ms for 180°. In terms of duty cycle, this corresponds to 5%, 7.5%, and 10% respectively. However, for the servo used in this lab, the calibrated duty cycles were found to be 2% for 0°, 7% for 90°, and 12% for 180°. A linear equation $y=(1/18)x+2$ can be used to calculate the duty cycle (y) for any desired angle (x). During the experiment, the servo motor did not rotate based on the input from the ultrasonic sensor. The issue was traced to the servo motor itself and was resolved by replacing it with a new one. This experience highlights the

importance of checking all hardware components, as a faulty part can prevent the system from working correctly.